Tutorial - 3

Searching and Sorting

Evaluative Questions

Q1. An *mxn* array A of real numbers is a Monge array if for all i, j, k, and l such that 1 <= i < k <= m and 1 <= j < l <= n, we have

A[i,j] + A[k, l] <= A[i,l] + A[k,j]

In other words, whenever we pick two rows and two columns of a Monge array and consider the four elements at the intersections of the rows and the columns, the sum of the upper-left and lower-right elements is less than or equal to the sum of the lower-left and upper-right elements.

Here is a description of a divide-and-conquer algorithm that computes the leftmost minimum element in each row of an m x n Monge array A:

a) Construct a submatrix A' of A consisting of the even-numbered rows of A. Recursively determine the leftmost minimum for each row of A'. Then compute the leftmost minimum in the odd-numbered rows of A.

Explain how to compute the leftmost minimum in the odd-numbered rows of A (given that the leftmost minimum of the even-numbered rows is known) in O(m+n) time.

b) Write the recurrence for the running time of the algorithm above. Show that its solution is O(m + n log m).

Solution

a)

Linearly scan row 1 indices 1 through $f(2)$ for the minimum element of row 1 and record as $f(1)$. Next linearly scan indices $f(2)$ through $f(4)$ of row 3 for the minimum element of row 3. In general, we need only scan indices $f(2k)$ through $f(2k+2)$ of row $2k+1$ to find the leftmost minimum element of row $2k+1$. If m is odd, we'll need to search indices $f(m-1)$ through n to find the leftmost minimum in row m. By part (c) we know that the indices of the leftmost minimum elements are increasing, so we are guaranteed to find the desired minimum from among the indices scanned. An element of column j will be scanned N_j+1 times, where N_j is the number of i such that $f(i) = j$. Since $\sum_{j=1}^{n} N_j = n$, the total number of comparisons is $m+n$, giving a running time of $O(m+n)$.

b)

Let $T(m,n)$ denote the running time of the algorithm applied to an m by n matrix. $T(m,n) = T(m/2,n) + c(m+n)$ for some constant c. We'll show

$$T(m,n) \le c(m + n\log m) - 2cm.$$

$$\begin{aligned} T(m,n) &= T(m/2,n) + c(m+n) \\ &\le c(m/2 + n\log(m/2)) - 2cm + c(m+n) \\ &= c(m/2 + n\log m) - cn + cn - cm \\ &\le c(m + n\log m) - cm \end{aligned}$$

so by induction we have $T(m,n) = O(m + n\log m)$.

Q7. Insertion sort on small arrays in merge sort
Although merge sort runs in O(n lg n) worst-case time and insertion sort runs in O(n^2) worst-case time, the constant factors in insertion sort can make it faster in practice for small problem sizes on many machines. Thus, it makes sense to coarsen the leaves of the recursion by using insertion sort within merge sort when subproblems become sufficiently small. Consider a modification to merge sort in which n/k sublists of length k are sorted using insertion sort and then merged using the standard merging mechanism, where k is a value to be determined.

a. Show that insertion sort can sort the n/k sublists, each of length k, in O(nk) worst-case time.

b. Show how to merge the sublists in O(n*lg(n/k)) worst-case time.

c. Given that the modified algorithm runs in O(nk + n*lg(n/k)) worst-case time, what is the largest value of k as a function of n for which the modified algorithm has the same running time as standard merge sort, in terms of O - notation?

Solutions:

a. The time for insertion sort to sort a single list of length k is $\Theta(k^2)$, so, n/k of them will take time $\Theta(\frac{n}{k}k^2) = \Theta(nk)$.

b. Suppose we have coarseness k. This meas we can just start using the usual merging procedure, except starting it at the level in which each array has size at most k. This means that the depth of the merge tree is $\lg(n) - \lg(k) = \lg(n/k)$. Each level of merging is still time cn, so putting it together, the merging takes time $\Theta(n \lg(n/k))$.

c. Viewing k as a function of n, as long as $k(n) \in O(\lg(n))$, it has the same asymptotics. In particular, for any constant choice of k, the asymptotics are the same.

d. If we optimize the previous expression using our calculus 1 skills to get k, we have that $c_1 n - \frac{nc_2}{k} = 0$ where c_1 and c_2 are the coeffients of nk and $n \lg(n/k)$ hidden by the asymptotics notation. In particular, a constant choice of k is optimal. In practice we could find the best choice of this k by just trying and timing for various values for sufficiently large n.

Practice Questions:

Q1. Suppose you are provided with the following function declaration in the C programming Language. (Quick Sort)

int partition(int a[], int n);

The function treats the first element of a[ ] as a pivot and rearranges the array so that all elements less than or equal to the pivot are in the left part of the array, and all elements greater than the pivot are in the right part. In addition, it moves the pivot so that the pivot is the last element of the left part. The return value is the number of elements in the left part. The following partially given function in the C programming language is used to find the kth smallest element in an array a[ ] of size n using the partition function. We assume k≤n.

```
int kth_smallest (int a[], int n, int k)
{
        int left_end = partition (a, n);
        if (left_end+1==k) {
                return a[left_end];
        }
        if (left_end+1 > k) {
        return kth_smallest (____________);
        } else {
        return kth_smallest (____________);
        }
```

}

The missing arguments lists are respectively

A. (a,left_end,k) and (a+left_end+1,n-left_end-1,k-left_end-1)

B. (a,left_end,k) and (a,n-left_end-1,k-left_end-1)

C. (a+left_end+1,n-left_end-1,k-left_end-1) and (a,left_end,k)

D. (a,n-left_end-1,k-left_end-1) and (a,left_end,k)

Answer: A.

We have to find the kth smallest element.

if (left_end+1 > k)

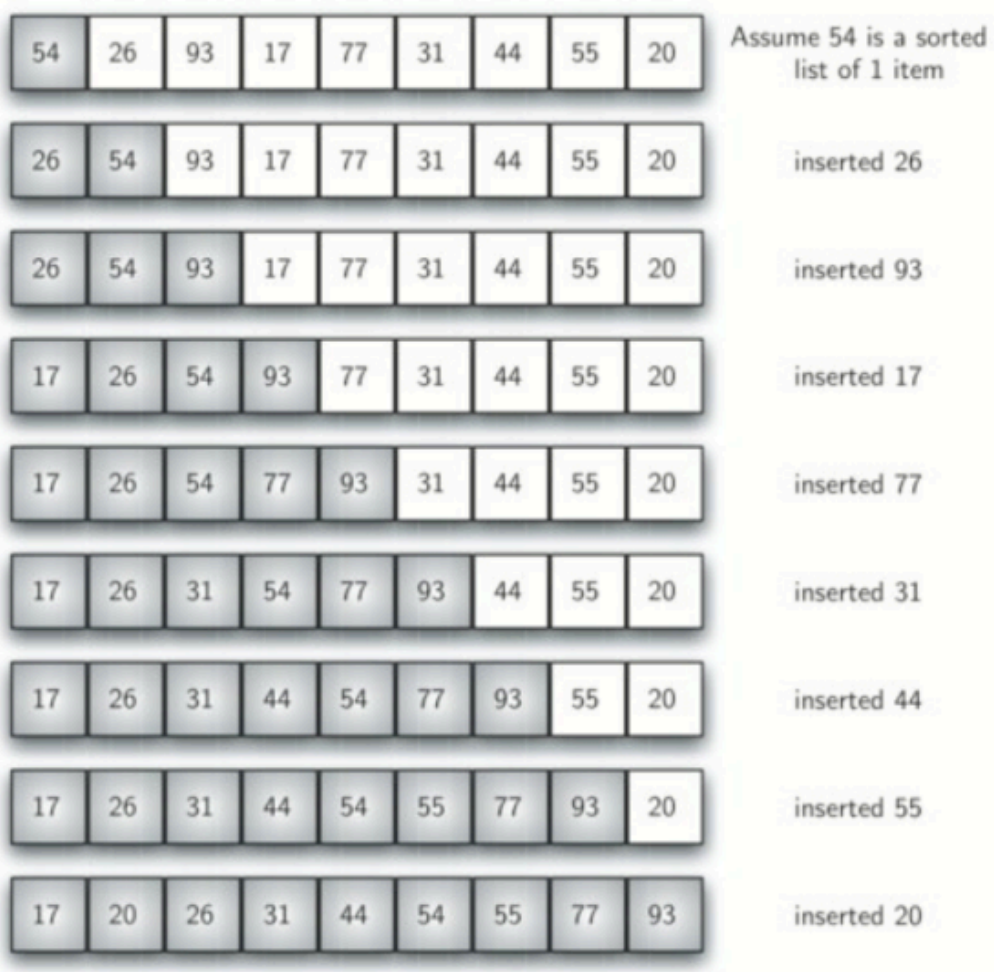
If the above condition is true, it means we have kth smallest element on the left array, whose size is left_end instead of n for the original array. (The "+1" is used because the array index in C language starts from 0). So, we can do a recursive call as

kth_smallest(a, left_end, k);

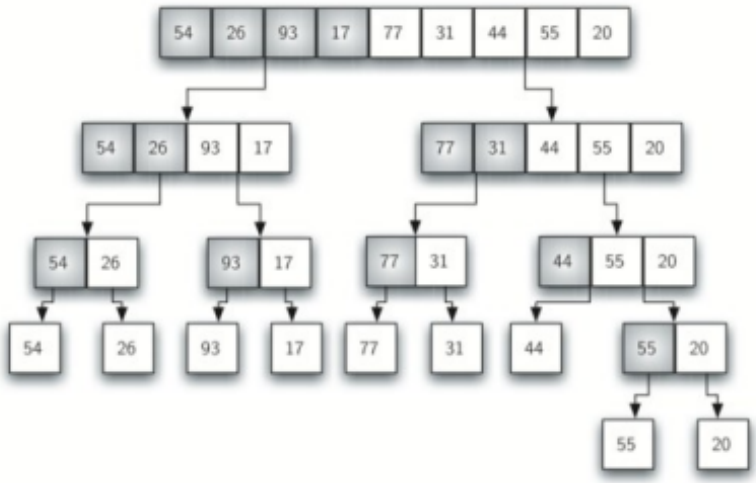
If the above condition is false, and left_end+1≠k, it means kth smallest element is on the right part of the array and it will be (k−left_end−1)th element there as left_end+1 elements are gone in the left part. So, the recursive call will be kth_smallest

(a + left_end + 1, n - left_end – 1, k - left_end - 1);

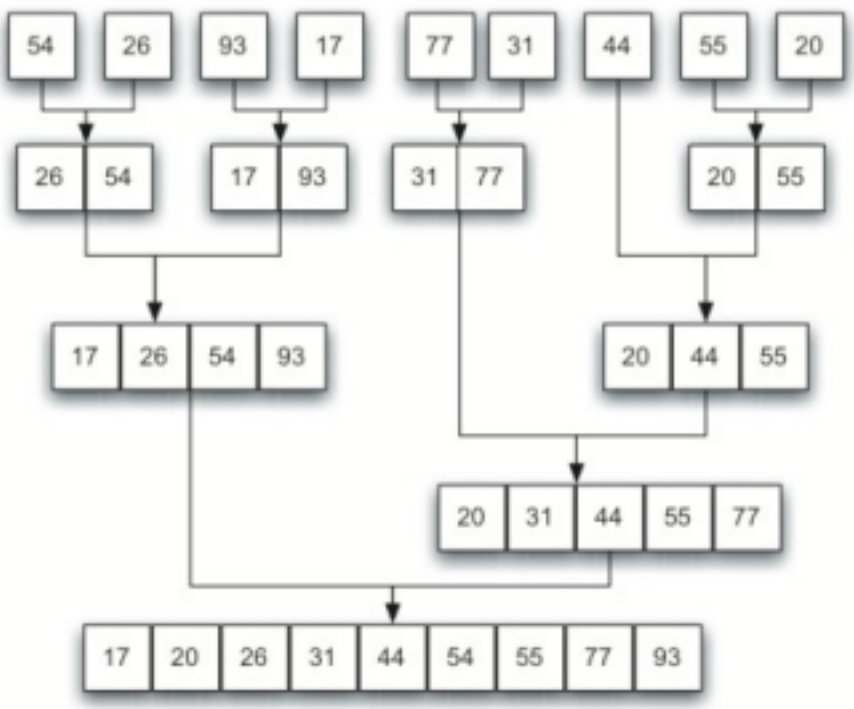
Q2. Demonstrate the insertion sort results for each insertion for the following initial array of elements . 54 26 93 17 77 31 44 55 20



Q3. Demonstrate the insertion sort results for each insertion for the following initial array of elements . 54 26 93 17 77 31 44 55 20



Splitting the image



Q5. Explain the "divide and conquer" strategy employed by the Mergesort algorithm. How does Mergesort recursively divide the array, and why is the merging step crucial for achieving a sorted result? Provide a step-by-step illustration of Mergesort on a small array.

Ans. Mergesort divides the array into two halves recursively until each subarray contains a single element. The merging step then combines these sorted subarrays to produce a fully sorted array. The merging step is crucial as it ensures that the combined elements maintain their sorted order.

Illustration: [4, 2, 7, 1] -> [4, 2], [7, 1] -> [4], [2], [7], [1] -> [2, 4], [1, 7] -> [1, 2, 4, 7].

Q6. (CLRS 4.2)

Use Strassen's algorithm to compute the matrix product

$$\begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}\begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}.$$

Show your work.

Solution:

$$\begin{aligned}
S_1 &= 8 - 2 = 6 \\
S_2 &= 1 + 3 = 4 \\
S_3 &= 7 + 5 = 12 \\
S_4 &= 4 - 6 = -2 \\
S_5 &= 1 + 5 = 6 \\
S_6 &= 6 + 2 = 8 \\
S_7 &= 3 - 5 = -2 \\
S_8 &= 4 + 2 = 6 \\
S_9 &= 1 - 7 = -6 \\
S_{10} &= 6 + 8 = 14
\end{aligned}$$

$$\begin{aligned}
P_1 &= 6 \\
P_2 &= 8 \\
P_3 &= 72 \\
P_4 &= -10 \\
P_5 &= 48 \\
P_6 &= -12 \\
P_7 &= -84
\end{aligned}$$

$$C_{11} = 48 - 10 - 8 - 12 = 18$$
$$C_{12} = 6 + 8 = 14$$
$$C_{21} = 72 - 10 = 62$$
$$C_{22} = 48 + 6 - 72 + 84 = 66$$

So, we get the final result:

$$\begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}$$

Q7. Suppose array A is an array of integers sorted in descending order. Our goal is to sort array A in ascending order. We have two choices to sort array A in ascending order.
The first choice is Quicksort, and the second choice is Mergesort. Which of the two sorting algorithms is better for this job and Why? Justify your answer.

Ans. Quicksort partitions the array around a pivot, dividing it into two subarrays. Mergesort, on the other hand, divides the array into two halves without a specific pivot. Quicksort's efficiency heavily relies on choosing a good pivot, while Mergesort consistently divides the array in half. Quicksort has an average-case time complexity of O(n log n), and Mergesort has a consistent O(n log n) time complexity.

Q8. Professor Caesar wishes to develop a matrix multiplication algorithm that is asymptomatically faster than Strassen's algorithm. This algo should use divide and conquer method, dividing each matrix into pieces of size n/4 X n/4 , and the divide and combine steps together will take O(n^2) time. How many sub-problems it should create for this to happen?
Solution:
Strassen's algorithm's running time is O(n^(lg 7)).
This algorithm's time complexity is
T(n) = a.T(n/4) +O(n^2) ; where 'a' is the number of subproblems.
We have to choose an 'a' s.t.
$\text{Log}_4 a < \lg 7$.
Therefore , a <= 48

Q9. Define a new "logarithmic factorial" function L(n) with the recurrence relation L(n) = L(n/2) + log(n). Apply the Master Theorem to analyze the time complexity of calculating L(n) and compare it to the standard factorial function. How does the growth rate behave differently, and what are the implications for large values of n?

Q10. Use mathematical induction to show that when n is an exact power of 2, the solution of the recurrence

$$T(n) = \begin{cases} 2 & \text{if } n = 2\,, \\ 2T(n/2) + n & \text{if } n = 2^k, \text{ for } k > 1 \end{cases}$$

is $T(n) = n \lg n$.

Solution:

Since n is a power of two, we may write $n = 2^k$. If $k = 1$, $T(2) = 2 = 2\lg(2)$. Suppose it is true for k, we will show it is true for $k+1$.

$$T(2^{k+1}) = 2T\left(\frac{2^{k+1}}{2}\right) + 2^{k+1} = 2T\left(2^k\right) + 2^{k+1} = 2(2^k \lg(2^k)) + 2^{k+1}$$

$$= k2^{k+1} + 2^{k+1} = (k+1)2^{k+1} = 2^{k+1}\lg(2^{k+1}) = n\lg(n)$$